Recap — C+ Setup: Kubernetes Lab

এক নজরে

আপনার Mac এ এখন একটা ৩-node Kubernetes cluster (`kind-subtrack-lab` , Kubernetes v1.37.0) চলছে, আর সব config একটা Git repo তে রাখা — এক command এ পুরো lab আবার বানানো যায়। এই doc টা পরে আবার পড়ার জন্য আর অন্য laptop এ নিজে setup করার জন্য।

যা install ও তৈরি হলো

জিনিস	Version / মান	কোথায়
macOS	12.7.6 Monterey, Intel (x86_64)	—
Docker Desktop	4.41.2 (engine 28.1.1) — **update করা যাবে না**	`/Applications/Docker.app`
Docker এর ভাগে resource	8 CPU, ~8 GB RAM	Docker Desktop → Settings → Resources
kubectl	v1.37.0	`~/.local/bin/kubectl`
kind	v0.33.0	`~/.local/bin/kind`
Helm	v4.3.0	`~/.local/bin/helm`
kube-ps1	—	`~/.kube-ps1/`
Cluster	`subtrack-lab` : ১ control-plane + ২ worker, Kubernetes v1.37.0	kubectl context `kind-subtrack-lab`
Namespaces	`lab` (default), `subtrack-dev` , `subtrack-prod`	cluster এর ভেতরে

আপনার বানানো file

File	কাজ
`~/k8s-lab/cluster/kind-config.yaml`	cluster দেখতে কেমন হবে (৩ node, image digest, port mapping)
`~/k8s-lab/cluster/namespaces.yaml`	৩টা namespace এর বর্ণনা
`~/k8s-lab/scripts/reset-lab.sh`	পুরো lab মুছে আবার বানানোর script
`~/k8s-lab/.gitignore`	secret জাতীয় file Git এ যাওয়া আটকায়
`~/.zshrc` এর `# >>> k8s-lab >>>` block (লাইন ১৫৪–১৬৪)	PATH, Tab-completion, `k` alias, kube-ps1
`~/.kube/config`	kubectl এর contact list — kind নিজে লিখেছে

Git এ ২টা commit: `f37018f` (lab setup) আর `94310f2` (reset script এ সব node Ready হওয়ার অপেক্ষা)।

বড় ছবি

পুরো setup টা ৬টা ধাপ, আর এর বেশিরভাগ প্রতি laptop এ একবারের কাজ — প্রতিদিন শুধু Docker চালু আছে কিনা আর কোন cluster এ আছেন, এটুকু দেখতে হয়।

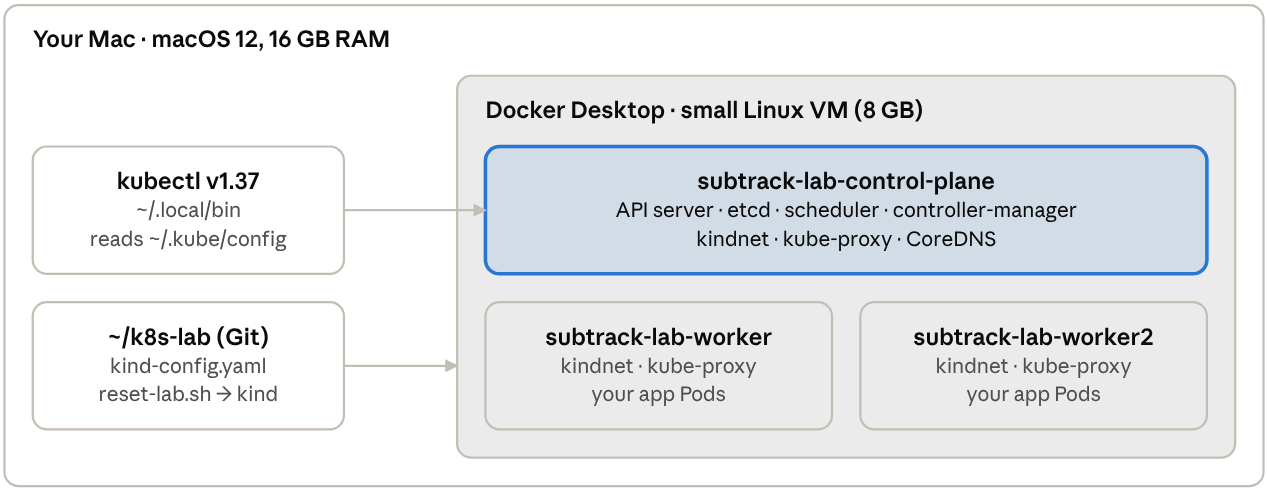
Building এর উদাহরণ

ধরুন practice এর জন্য একটা ছোট apartment building বানাচ্ছেন।

বাস্তব জিনিস	Building এ	কাজ
Docker Desktop	জমি আর ভিত	Mac এর ভেতরে একটা ছোট Linux computer চালায়; সব কিছু এর উপর বসে
kind	ঠিকাদার	Docker এর উপর ৩টা "ফ্ল্যাট" (node) বানিয়ে দিয়ে চলে যায়
Kubernetes cluster	management office + ফ্ল্যাটগুলো	কোন app কোন ফ্ল্যাটে থাকবে, কেউ মারা গেলে নতুন আনা — সামলায়
kubectl	আপনার ফোন	office কে নির্দেশ দেন: "এটা চালাও", "এটা দেখাও"

বাস্তব জিনিস	Building এ	কাজ
kubeconfig (`~/.kube/config`)	ফোনের contact list	কোন office এর নম্বর কোনটা
Helm	ready-made ফার্নিচারের প্যাকেজ	পরে লাগবে (P6 থেকে)

The whole cluster is three containers inside Docker on your Mac



আপনার lab · Mac → Docker VM → ৩টা node container

kubectl laptop থেকে HTTPS এ control-plane container এর ভেতরের API server এ কথা বলে; Docker বন্ধ হলে ভেতরের তিনটা node ই থেমে যায়, তাই `connection refused` ।

৬টা ধাপ, আর কোনটা কতবার

ধাপ	কী করলাম	কেন	কতবার
১. Docker	চালু, RAM/CPU ঠিক	kind এর node গুলো Docker container	install প্রতি laptop এ একবার; চালু থাকতে হবে প্রতিবার
২. Tools	kubectl, kind, Helm	cluster বানানো ও চালানোর হাতিয়ার	প্রতি laptop এ একবার (মাঝে মাঝে update)
৩. Shell	PATH, Tab-completion, prompt	সঠিক kubectl চলা আর ভুল cluster এড়ানো	প্রতি laptop এ একবার

ধাপ	কী করলাম	কেন	কতবার
৪. Cluster	kind config লিখে cluster	এখানেই Kubernetes চলে	একবার; ভাঙলে আবার
৫. Environment	namespace, Git, reset script	গুছিয়ে কাজ আর এক command এ সব ফেরত	cluster বানানোর পর প্রতিবার — script নিজে করে
৬. প্রতিদিন	Docker চালু? কোন cluster?	—	প্রতিদিন কাজ শুরুর আগে

ভাগ ১ — Docker আর tools

Docker না চললে cluster ও চলে না, আর kubectl এর version cluster এর ১ minor version এর মধ্যে থাকতে হয় — এই দুটোই এই ভাগের মূল কথা।

Mac এ Docker Desktop কেন লাগে

Container চালাতে Linux kernel লাগে, macOS Linux না। Docker Desktop তাই Mac এর ভেতরে একটা ছোট Linux VM চালায়, আর container গুলো তার ভেতরে চলে। kind এর ৩টা node ও এই VM এর ভেতরের container।

Docker এর দুটো অংশ: `docker` command হলো **client**, আর যে program আসলে container চালায় সেটা **server (daemon)**। Client একটা socket file দিয়ে server এর সাথে কথা বলে। kubectl (client) আর API server (server) ও ঠিক একইভাবে কাজ করে।

Command	কী করে	Output এর মানে
`open -a Docker`	Docker Desktop চালু	menu bar এ তিমি icon
`docker version`	client ও server এর version	`Server:` খালি বা `Cannot connect to the Docker daemon` → Docker চালু নেই
`docker info --format '{{.NCPU}} {{.MemTotal}}'`	Docker কত CPU/RAM পেয়েছে	kind এর সব node এই ভাগের মধ্যে চলে
`docker run --rm hello-world`	test container চালিয়ে মুছে দেয়	`Hello from Docker!` → image download আর container চালানো দুটোই ঠিক

Command	কী করে	Output এর মানে
`docker ps`	চলমান container	kind এর ৩টা node এখানে দেখা যায়

Resource কেন 8 GB: ৩টা node আর পরে আপনার app — সবাই এই ভাগ থেকে খায়। কম দিলে cluster অচল, বেশি দিলে Mac ধীর। 16 GB এর অর্ধেক ভারসাম্যপূর্ণ।

তিনটা tool

Tool	কাজ	কতবার ব্যবহার
kubectl	cluster এর remote control	প্রতিদিন
kind	cluster বানানো ও মোছা	মাঝেমধ্যে
Helm	ready-made package install	P6 থেকে

Install এর পেছনের চারটা সিদ্ধান্ত

1. **আগে `/tmp` এ পরীক্ষা:** macOS 12 পুরনো, নতুন tool চলবে কিনা জানা ছিল না। `/tmp` এ download করে `kind version` চালিয়ে দেখলাম, তারপর install। অনুমান না করে আগে পরীক্ষা।
2. **Checksum যাচাই:** প্রতিটা file এর একটা আঙুলের ছাপ (SHA-256 hash) থাকে। Official hash এর সাথে মিললে `kubectl: OK` আসে। `&&` মানে "আগেরটা সফল হলে তবেই পরেরটা" — তাই না মিললে install হতোই না।
3. **`~/.local/bin` এ install:** `/usr/local/bin/kubectl` এ Docker Desktop এর নিজের পুরনো kubectl (v1.32, একটা symlink বা shortcut) আছে, সেটা ভাঙিনি। `~/.local/bin` আপনার নিজের folder, `sudo` লাগে না। `install -m 0755` file কপি করে আর চালানোর অনুমতি দেয়।
4. **kubectl v1.37:** নিয়ম হলো kubectl আর cluster এর পার্থক্য সর্বোচ্চ ১ minor version। Cluster v1.37, তাই kubectl লাগবে v1.36 থেকে v1.38।

আপনার Mac এর বিশেষ কথা

- **Docker Desktop update করবেন না।** Docker শুধু macOS এর current আর আগের দুটো major version support করে; Monterey (12) সেই তালিকায় নেই।
- **Homebrew ব্যবহার করিনি।** macOS 12 এ Homebrew এর support নেই; তাই official binary সরাসরি নিয়েছি।
- **Helm install করা হয়েছে `--no-sudo` দিয়ে** official `get-helm-4` script থেকে; script নিজে checksum যাচাই করে।

ভাগ ২ — Shell, PATH আর prompt

"কোন kubectl চলছে" সন্দেহ হলে প্রথম command সবসময় `whence -a kubectl` — PATH এর ক্রম আর zsh এর মনে রাখা, এই দুটোই এর উত্তর ঠিক করে।

Terminal আর shell আলাদা

জিনিস	আপনার ক্ষেত্রে	কাজ
Terminal	Warp	window — লেখা দেখায়, keyboard থেকে নেয়
Shell	zsh	ভেতরের program — command বোঝে আর চালায়

PATH — shell কীভাবে command খুঁজে পায়

PATH হলো `:` দিয়ে আলাদা করা folder এর তালিকা। zsh বাম থেকে ডানে খোঁজে, **প্রথম যেখানে পায় সেটাই চালায়**। বামে থাকা মানে বেশি priority।

লেখার ধরন	folder বসে	Priority
`export PATH="$HOME/.local/bin:$PATH"`	শুরুতে	সবচেয়ে বেশি
`export PATH="$PATH:$HOME/.local/bin"`	শেষে	সবচেয়ে কম

`$PATH` মানে "আগের পুরো তালিকা"। `.zshrc` উপর থেকে নিচে চলে, তাই যে লাইন **পরে** কিছু শুরুতে বসায়, সেটা সামনে চলে আসে। ক্রম তখনই গুরুত্বপূর্ণ যখন একই নামের program দুই folder এ থাকে।

```
echo $PATH | tr ':' '\n'   # PATH এর folder গুলো এক লাইনে একটা করে
whence -a kubectl          # যত জায়গায় kubectl আছে, ক্রম অনুসারে
grep -n 'PATH' ~/.zshrc    # .zshrc এর কোন লাইনে PATH বদলানো হয়েছে
```

নতুন tool এলে: সবচেয়ে সহজ — binary টা `~/.local/bin` এ রাখুন (`install -m 0755 tool ~/.local/bin/tool` , তারপর `rehash`)। Tool নিজের folder চাইলে `.zshrc` এর একদম শেষে `export PATH="<folder>:$PATH"` যোগ করে নতুন tab খুলুন।

অস্থায়ী বনাম স্থায়ী: terminal এ সরাসরি `export` শুধু ওই tab এ; `.zshrc` এ লিখলে সব নতুন tab এ।

`.zshrc` — প্রতিটা নতুন tab এর শুরুর নির্দেশনা

নতুন tab খুললে zsh `~/.zshrc` উপর থেকে নিচে চালায়। চলমান tab নিজে থেকে আবার পড়ে না — তাই বদলানোর পর নতুন tab, বা `source ~/.zshrc` ।

zsh এর মনে রাখা (hash) — সেদিনের রহস্য

zsh একবার command খুঁজে পেলে তার জায়গা মনে রাখে। ওই tab এ আগে পুরনো kubectl চালানো হয়েছিল, তাই নতুন install এর পরেও `which` ঠিক উত্তর দিলেও চলছিল পুরনোটা। `rehash` zsh কে মনে রাখা তালিকা ভুলে যেতে বলে। শিক্ষা: **নতুন tool install বা update এর পর নতুন tab** (বা `rehash`)।

আমাদের `.zshrc` block

```
# >>> k8s-lab >>>
typeset -U path PATH                    # PATH এ একই folder দুবার থাকলে একটা বাদ
autoload -Uz compinit && compinit       # zsh এর Tab-completion চালু
source <(kubectl completion zsh)        # kubectl এর Tab-completion
source <(helm completion zsh)           # helm এর Tab-completion
alias k=kubectl                         # "k" লিখলেই kubectl
compdef __start_kubectl k               # "k" তেও Tab-completion
source ~/.kube-ps1/kube-ps1.sh          # kube-ps1 load
setopt PROMPT_SUBST
PROMPT='$(kube_ps1) '$PROMPT            # prompt এর শুরুতে cluster:namespace
# <<< k8s-lab <<<
```

শুরু আর শেষের চিহ্ন দিয়ে block চেনা যায়; পুরোটা মুছলে সব আগের মতো।

Warp এর prompt

Warp নিজের prompt নিজে বানায় (`PROMPT` এর মান `_warp_stripped_prompt`, আর `warp_precmd` প্রতিটা command এর আগে সেটা আবার বসায়)। তাই kube-ps1 Warp এ দেখা যায় না; macOS Terminal বা VS Code terminal এ দেখা যাবে।

Warp এ একই কাজ করে **`kube-context` chip** (Settings → Appearance → Input → prompt preview → Edit prompt) — `kubectl`, `helm`, `k9s` লেখার সময় cluster এর নাম দেখায়। Chip namespace দেখায় না; namespace জানতে:

```
kubectl config view --minify -o jsonpath='{..namespace}'; echo
```

পরিস্থিতি	কী করবেন
নতুন laptop	`~/.local/bin`, PATH, `.zshrc` block — একবার
tool install বা update	নতুন tab, বা `rehash`
`.zshrc` বদলেছেন	নতুন tab, বা `source ~/.zshrc`

পরিস্থিতি	কী করবেন
কোন version চলছে সন্দেহ	`whence -a kubectl` + `kubectl version --client`

ভাগ ৩ — Cluster বানানো

Cluster এর পুরো বর্ণনা একটা ছোট YAML file এ, আর `kind create cluster` সেটা পড়ে ১–২ মিনিটে ৩টা node দাঁড় করায়।

`kind-config.yaml` — লাইন ধরে

```
kind: Cluster                              # ① এটা একটা Cluster এর বর্ণনা
apiVersion: kind.x-k8s.io/v1alpha4         # ② kind এর config format এর version
name: subtrack-lab                         # ③ cluster এর নাম
nodes:                                     # ④ কয়টা node, কোনটা কী ভূমিকায়
- role: control-plane
  image:
kindest/node:v1.37.0@sha256:a1ed56cfb0e7b93589bdf97c8cd566405a265939e3620fc4f
5de89adff580ae5
  extraPortMappings:                       # ⑥ laptop থেকে ভেতরে ঢোকার দরজা
  - containerPort: 30080
    hostPort: 30080
  - containerPort: 30443
    hostPort: 30443
- role: worker
  image:
kindest/node:v1.37.0@sha256:a1ed56cfb0e7b93589bdf97c8cd566405a265939e3620fc4f
5de89adff580ae5
- role: worker
  image:
kindest/node:v1.37.0@sha256:a1ed56cfb0e7b93589bdf97c8cd566405a265939e3620fc4f
5de89adff580ae5
```

- ① ② Kubernetes এর প্রতিটা YAML ও `kind` আর `apiVersion` দিয়ে শুরু হয় (M1.6); এই file kind এর, কিন্তু লেখার ধরন একই।
- ③ **`name`** থেকে context হয় `kind-subtrack-lab`, node হয় `subtrack-lab-worker`। নাম না দিলে সব `kind` হতো।

- ④ **৩টা node:** control plane = management office (API server, etcd, scheduler, controller-manager); ২টা worker এ app চলবে। ২টা worker থাকায় পরে দেখা যাবে scheduler কীভাবে Pod ভাগ করে আর node মারা গেলে Pod কোথায় যায়।
- ⑤ **`@sha256` digest:** `v1.37.0` tag পরে অন্য image কে দেখাতে পারে; digest কখনো বদলায় না। তাই অন্য laptop বা ৬ মাস পরেও একই cluster। Digest নেওয়া kind v0.33.0 release notes থেকে।
- ⑥ **`extraPortMappings`:** node গুলো Docker container, তাদের port laptop থেকে সরাসরি পৌঁছায় না। laptop এর 30080 → control-plane এর 30080। M3.1 এ NodePort এর জন্য লাগবে; cluster **বানানোর সময়ই** দিতে হয়।

`kind create cluster` এর output

```
kind create cluster --config ~/k8s-lab/cluster/kind-config.yaml --wait 5m
```

Output	ভেতরে কী হচ্ছে
`Ensuring node image`	node image download (প্রথমবার কয়েক মিনিট, পরে cache থেকে)
`Preparing nodes`	৩টা Docker container চালু = ৩টা node
`Writing configuration`	Kubernetes চালুর settings প্রতিটা node এ
`Starting control-plane`	API server, etcd, scheduler, controller-manager চালু
`Installing CNI`	**kindnet** — Pod কে IP দেয়, node গুলোর মধ্যে network জোড়ে
`Installing StorageClass`	**standard** — পরে app storage চাইলে disk বানায় (M4.2)
`Joining worker nodes`	২টা worker যোগ দিল
`Waiting ... control-plane = Ready`	`--wait` শুধু control plane এর জন্য অপেক্ষা করে, worker দের জন্য না
`Set kubectl context to "kind-subtrack-lab"`	kind `~/.kube/config` এ ঠিকানা লিখে current করল

যাচাই

Command	কী খেয়াল করবেন
`kubectl config current-context`	`kind-subtrack-lab`
`kubectl cluster-info`	API server `https://127.0.0.1:<port>` — laptop এর port থেকে control-plane container এর ভেতরের API server এ
`kubectl get nodes -o wide`	৩টা `Ready`, VERSION `v1.37.0`, runtime `containerd`
`kubectl get pods -n kube-system -o wide`	সব `Running`
`docker ps`	৩টা container = ৩টা node

Smoke test

```
kubectl run hello --image=nginx:1.27 --restart=Never             # একটা nginx Pod
kubectl wait --for=condition=Ready pod/hello --timeout=180s       # প্রস্তুত হওয়া পর্যন্ত অপেক্ষা
kubectl get pod hello -o wide                                     # কোন node, কোন IP
kubectl port-forward pod/hello 8080:80 >/dev/null 2>&1 &          # laptop 8080 → Pod 80 সুড়ঙ্গ, পেছনে
sleep 3; curl -s localhost:8080 | grep -o '<title>.*</title>'     # Welcome to nginx!
kill %1                                                           # সুড়ঙ্গ বন্ধ
kubectl delete pod hello                                          # পরিষ্কার
```

`Welcome to nginx!` এলে প্রমাণ হয়: scheduler node বেছেছে, image এসেছে, Pod চলছে, network কাজ করছে, kubectl থেকে Pod পর্যন্ত পৌঁছানো যায়।

Output থেকে যা শিখলাম

যা দেখলাম	মানে	কোথায় আবার আসবে
Node IP `172.18.0.x`, Pod IP `10.244.x.x`	node এর network Docker এর; Pod এর network CNI এর। তৃতীয় range Service এর	M3.3
প্রতিটা node আলাদা Pod range (`10.244.0.x`, `10.244.3.x`)	CNI প্রতিটা node কে Pod network এর একটা ভাগ দেয়	M3.3
etcd, apiserver, scheduler এর IP = node এর IP, নামের শেষে node এর নাম	static Pod — kubelet সরাসরি চালায়, node এর network ব্যবহার করে	M1.2
`kindnet`, `kube-proxy` ঠিক ৩টা করে	DaemonSet — প্রতিটা node এ একটা	M7.3
CoreDNS এর দুটো Pod ই control-plane এ	scheduler সিদ্ধান্ত একবারই নেয়; পরে node এলে চলমান Pod সরে না	M5.4
তিনটা node এর kernel একই `linuxkit`	container kernel share করে — VM এর সাথে মূল পার্থক্য	M0.4
Runtime `containerd`	Kubernetes এখন Docker Engine সরাসরি ব্যবহার করে না	M1.2
StorageClass `standard` : `Delete`, `WaitForFirstConsumer`	PVC মুছলে data যায়; Pod এর node ঠিক হওয়ার পর disk	M4.2
Reset এর পর node এর IP বদলেছে	IP স্থায়ী না — কখনো hardcode না, নাম আর Service ব্যবহার	M3.1, M3.2

ভাগ ৪ — Namespace, Git আর reset script

সব কিছু file এ লিখে Git এ রাখা হয়েছে, তাই `./scripts/reset-lab.sh` এক command এ cluster, namespace আর default namespace হুবহু ফেরত আনে — একে বলে declarative পদ্ধতি বা "Infrastructure as Code"।

Namespace — cluster এর ভেতরের folder

Namespace	কাজ
`lab`	রোজকার পরীক্ষা — নিশ্চিন্তে মুছতে পারবেন
`subtrack-dev`	SubTrack project এর development জগৎ
`subtrack-prod`	production এর মতো কড়া নিয়মের জগৎ (P8 থেকে)

`~/k8s-lab/cluster/namespaces.yaml` :

```
apiVersion: v1
kind: Namespace
metadata:
  name: lab
  labels:
    purpose: scratch
---
apiVersion: v1
kind: Namespace
metadata:
  name: subtrack-dev
  labels:
    app.kubernetes.io/part-of: subtrack
    environment: dev
---
apiVersion: v1
kind: Namespace
metadata:
  name: subtrack-prod
  labels:
    app.kubernetes.io/part-of: subtrack
    environment: prod-sim
```

- **Command না, YAML কেন:** `kubectl create namespace` একবারের কাজ, কোথাও লেখা থাকে না। YAML Git এ থাকে, cluster মুছলেও `kubectl apply -f` দিয়ে হুবহু ফেরত।
- **`kubectl apply`** : না থাকলে বানায়, থাকলে update করে — বারবার চালালেও ক্ষতি নেই।
- **`---`** : এক file এ একাধিক YAML document আলাদা করে।
- **Label** (`environment: dev`): গায়ে লাগানো ট্যাগ — পরে খোঁজা আর নিয়ম বসাতে (M2.3, M8.4)। Kubernetes নিজে `kubernetes.io/metadata.name` label যোগ করে।

Default namespace

```
kubectl config set-context --current --namespace=lab
```

`~/.kube/config` এ context এর ভেতরে `namespace: lab` লেখে। এরপর `-n` না দিলে সব command `lab` এ যায়। প্রথম smoke test এর `hello` Pod `default` namespace এ মুছেছিল, কারণ তখনো এটা set হয়নি।

Git আর `.gitignore`

Cluster ক্ষণস্থায়ী, config স্থায়ী। `.gitignore` এর file গুলো Git কখনো নেয় না, তাই password বা token ভুল করে commit হয় না:

```
secrets/
*.env
*.kubeconfig
```

`reset-lab.sh` — লাইন ধরে

```
#!/usr/bin/env bash          # এই file bash দিয়ে চালাও
# পুরো lab cluster মুছে নতুন করে বানায় — ভেতরের সব data চলে যাবে
set -euo pipefail            # কোনো ধাপ fail করলে সাথে সাথে থামো
cd "$(dirname "$0")/.."      # যেখান থেকেই চালান, আগে k8s-lab folder এ যাও
kind delete cluster --name subtrack-lab                          # পুরনো
cluster মোছো
kind create cluster --config cluster/kind-config.yaml --wait 5m  # নতুন বানাও
kubectl apply -f cluster/namespaces.yaml                         # namespace
ফেরত
kubectl config set-context --current --namespace=lab             # default
namespace
kubectl wait --for=condition=Ready nodes --all --timeout=180s    # সব node
Ready পর্যন্ত অপেক্ষা
kubectl get nodes                                                # ফলাফল
```

`chmod +x scripts/reset-lab.sh` দিয়ে চালানোর যোগ্য করা হয়েছে, তাই `./scripts/reset-lab.sh` লিখলেই চলে। `kubectl wait` লাইনটা পরে যোগ করা হয়েছে, কারণ `--wait` শুধু control plane এর জন্য অপেক্ষা করে — প্রথম reset এ worker গুলো কয়েক সেকেন্ড `NotReady` ছিল।

পথে আসা সমস্যা ও সমাধান

আজ ১০টা সমস্যা এসেছিল, প্রতিটার সমাধান একই পদ্ধতিতে: লক্ষণ দেখা → একটা command দিয়ে কারণ প্রমাণ → ছোট পরিবর্তন → আবার যাচাই। এই পদ্ধতিই M10.1 এর troubleshooting।

#	লক্ষণ	কারণ	কীভাবে প্রমাণ / ঠিক করলাম	শিক্ষা
1	`Cannot connect to the Docker daemon`	Docker Desktop চালু ছিল না	`open -a Docker`, তারপর `docker version` এ Server এল	client চালু থাকলেই হয় না, server ও চালু থাকতে হয়
2	kubectl v1.32.2	Docker Desktop এর নিজের kubectl (symlink)	`ls -l $(which kubectl)` → Docker.app কে দেখায়; v1.37 `~/.local/bin` এ	অন্যের জিনিস না ভেঙে নিজের জায়গায় install
3	macOS 12 পুরনো	Docker Desktop আর Homebrew এর support নেই	official binary `/tmp` এ চালিয়ে পরীক্ষা, তারপর install	অনুমান না, আগে পরীক্ষা
4	`which` নতুন দেখায়, চলে পুরনো	zsh এর hash (মনে রাখা)	`rehash` বা নতুন tab	install/update এর পর নতুন tab
5	PATH এ `~/.local/bin` দুবার	আগে থেকেই ছিল, আবার যোগ হয়েছে	`typeset -U path PATH`	ক্ষতি নেই, তবে পরিষ্কার রাখা ভালো
6	prompt এ kube-ps1 নেই (`grep -c kube_ps1` = 0)	Warp নিজের prompt বসায়	`print -r -- "$PROMPT"` → `_warp_stripped_prompt`; Warp এর `kube-context` chip ব্যবহার	terminal app ও ব্যবহার বদলায়
7	`echo "$(kube_ps1)"` এ `%{%F{4}%}`	zsh prompt এর রঙের code	`print -P` দিয়ে রঙিন রূপ দেখা	echo code বোঝে না, prompt বোঝে
8	paste করা command এ বাংলা লেখা ভাঙা	terminal এ বাংলা paste এর display সমস্যা	`cat` দিয়ে file দেখে নিশ্চিত — file ঠিক	যা দেখছেন আর যা আছে আলাদা হতে পারে; যাচাই করুন

#	লক্ষণ	কারণ	কীভাবে প্রমাণ / ঠিক করলাম	শিক্ষা
9	`pod "hello" deleted from default namespace`	default namespace তখনো set হয়নি	`kubectl config set-context --current --namespace=lab`	namespace না বললে Pod "হারিয়ে" যায়
10	reset এর পরে worker `NotReady`	`--wait` শুধু control plane এর জন্য; worker এর kindnet তখনো চালু হচ্ছিল	`kubectl get pods -n kube-system -l app=kindnet -o wide`; script এ `kubectl wait ... nodes --all` যোগ	node Ready হতে CNI লাগে

প্রতিদিনের রুটিন ও নতুন laptop runbook

প্রতিদিন কাজ শুরুর আগে ৩টা command (৩০ সেকেন্ড); নতুন laptop এ ৬টা ধাপ (~১ ঘণ্টা)।

প্রতিদিন কাজ শুরুর আগে

```
open -a Docker       # Docker চালু (চালু থাকলে কিছু হবে না)
docker ps            # ৩টা node container "Up"?
kubectl get nodes    # ৩টা node Ready?
```

তারপর `kubectl` লেখার সময় Warp chip এ `kind-subtrack-lab` দেখে নিন। কাজ শেষে কিছু করার দরকার নেই; RAM খালি করতে Docker Desktop Quit করা যায়। পরের দিন cluster না ফিরলে `~/k8s-lab/scripts/reset-lab.sh`।

লক্ষণ	প্রথম কাজ
`connection refused`	`docker version` — Docker চালু?
Docker চালু, কিন্তু `docker ps` এ node নেই	`docker ps -a`, তারপর `reset-lab.sh`
node `NotReady`	`kubectl describe node <নাম>` এর Conditions, আর ওই node এর kindnet Pod
"কোন kubectl চলছে?"	`whence -a kubectl`

নতুন laptop এ পুরো setup (macOS)

Version গুলো Oct 2026 এর। নতুন kind নিলে release notes থেকে নতুন node image digest নেবেন, আর kubectl রাখবেন cluster এর ±১ minor version এ। Apple Silicon Mac হলে `ARCH=arm64`।

ধাপ ১ — Docker

Docker Desktop install (OS এর সাথে মেলে এমন version), চালু করে Settings → Resources এ 6–8 CPU, 8 GB RAM।

```
docker run --rm hello-world     # "Hello from Docker!"
```

ধাপ ২ — Tools (checksum সহ)

```
mkdir -p ~/.local/bin && cd /tmp
ARCH=amd64   # Apple Silicon হলে arm64

# kubectl v1.37.0
curl -fsSLo kubectl
https://dl.k8s.io/release/v1.37.0/bin/darwin/$ARCH/kubectl
curl -fsSLo kubectl.sha256
https://dl.k8s.io/release/v1.37.0/bin/darwin/$ARCH/kubectl.sha256
echo "$(cat kubectl.sha256)  kubectl" | shasum -a 256 --check && install -m
0755 kubectl ~/.local/bin/kubectl

# kind v0.33.0
curl -fsSLo kind https://kind.sigs.k8s.io/dl/v0.33.0/kind-darwin-$ARCH
curl -fsSLo kind.sha256sum https://github.com/kubernetes-
sigs/kind/releases/download/v0.33.0/kind-darwin-$ARCH.sha256sum
echo "$(awk '{print $1}' kind.sha256sum)  kind" | shasum -a 256 --check &&
install -m 0755 kind ~/.local/bin/kind

# Helm 4 (script নিজে checksum যাচাই করে)
curl -fsSLo get_helm.sh
https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-4
HELM_INSTALL_DIR="$HOME/.local/bin" bash get_helm.sh --no-sudo
```

`kubectl: OK` আর `kind: OK` না এলে থামুন।

ধাপ ৩ — Shell

```
grep -q '.local/bin' ~/.zshrc || echo 'export PATH="$HOME/.local/bin:$PATH"'
>> ~/.zshrc
[ -d ~/.kube-ps1 ] || git clone --depth 1 https://github.com/jonmosco/kube-
ps1.git ~/.kube-ps1
grep -q '# >>> k8s-lab >>>' ~/.zshrc || cat >> ~/.zshrc <<'EOF'
# >>> k8s-lab >>>
typeset -U path PATH
autoload -Uz compinit && compinit
source <(kubectl completion zsh)
source <(helm completion zsh)
alias k=kubectl
compdef __start_kubectl k
source ~/.kube-ps1/kube-ps1.sh
setopt PROMPT_SUBST
PROMPT='$(kube_ps1) '$PROMPT
# <<< k8s-lab <<<
EOF
```

নতুন tab খুলে যাচাই: `whence -a kubectl` এর প্রথম লাইন `~/.local/bin/kubectl`, আর `kubectl version --client` → v1.37.0। Warp ব্যবহার করলে Settings → Appearance → Input → Edit prompt এ `kube-context` chip আছে কিনা দেখুন।

ধাপ ৪ — `k8s-lab` repo আনা

পুরনো laptop থেকে `~/k8s-lab` folder কপি করুন (GitHub এর private repo তে push করা থাকলে `git clone`)।

ধাপ ৫ — Cluster আর namespace

```
cd ~/k8s-lab && ./scripts/reset-lab.sh
```

ধাপ ৬ — যাচাই

```
kubectl get nodes                                                  # ৩টা Ready
kubectl config view --minify -o jsonpath='{..namespace}'; echo  # lab
```

তারপর ভাগ ৩ এর smoke test চালান।

Linux বা Windows (WSL2) হলে: URL এ `darwin` এর জায়গায় `linux`, `shasum -a 256` এর জায়গায় `sha256sum`, আর Docker হবে Docker Engine (Linux) বা Docker Desktop + WSL Integration (Windows)। Roadmap doc এর C+ অংশে বিস্তারিত আছে।

Review প্রশ্ন ও self-test

চার ভাগের চারটা প্রশ্নের মধ্যে তিনটা আপনি সম্পূর্ণ ঠিক বলেছেন; ভাগ ৩ এর প্রশ্নে অনুমানটা যুক্তিসঙ্গত ছিল, তবে আসল কারণ ছিল CNI।

পাঠের সময়ের প্রশ্ন ও আপনার উত্তর

ভাগ	প্রশ্ন	আপনার উত্তর	পূর্ণ উত্তর
১	সকালে `kubectl get nodes` এ `connection refused` — প্রথমে কী চেক করবেন?	Docker চালু আছে কিনা, কারণ node গুলো Docker এর ভেতরে চলে	সঠিক। যাচাই: `docker version` (Server আছে?), `docker ps` (৩টা node Up?)
২	নতুন laptop এ v1.37 install করলেন, কিন্তু পুরনো version চলছে — দুটো কারণ?	১. zsh আগের জায়গা মনে রেখেছে (cache); ২. PATH এর ক্রম বদলায়নি	সঠিক। যাচাই দুটোতেই `whence -a kubectl`; সমাধান: ১ → `rehash` /নতুন tab, ২ → `.zshrc` এর শেষে `export PATH="$HOME/.local/bin:$PATH"`। তৃতীয় সম্ভাবনা: `~/.local/bin` PATH এ নেই-ই
৩	২ মিনিট পরেও একটা worker `NotReady` — কাকে সন্দেহ করবেন?	হয়তো অন্য কোনো version এ চলছে	যুক্তিসঙ্গত অনুমান, `kubectl get nodes -o wide` এর VERSION এ এক নজরে বাদ দেওয়া যায় (সব node একই digest)। আসল সন্দেহভাজন CNI (kindnet): `kubectl describe node <নাম>` এর Conditions, তারপর ওই node এর kindnet Pod
৪ক	নতুন laptop এ cluster আর namespace এক command এ?	`./scripts/reset-lab.sh`	সঠিক
৪খ	namespace YAML file দিয়ে কেন, `kubectl create` দিয়ে না?	command দিয়ে বানালে কোথাও save থাকে না, আবার বানাতে হয়; file থাকলে script চালালেই ফেরত আসে	সঠিক — এটাই declarative পদ্ধতি / Infrastructure as Code

Self-test — আগে নিজে উত্তর দিন, তারপর নিচে মেলান

1. `Cannot connect to the Docker daemon` এর মানে কী?
2. `which kubectl` নতুন version দেখাল, কিন্তু চলল পুরনোটা — কেন, আর সমাধান?
3. kubectl কীভাবে জানে কোন cluster এ যাবে? Context এর তিনটা অংশ কী?
4. Node image কেন `@sha256` digest দিয়ে pin করলাম?
5. Pod IP (`10.244.x.x`) আর node IP (`172.18.0.x`) আলাদা কেন, কে দেয়?
6. Reset এর পরে worker কিছুক্ষণ `NotReady` ছিল কেন?
7. PATH এ একটা folder কে সবার আগে বসাতে কোন লাইন লিখবেন, আর কোথায়?
8. Warp এ kube-ps1 কেন দেখা যায় না, আর namespace কীভাবে জানবেন?

উত্তর

1. `docker` command (client) আছে, কিন্তু যে program container চালায় (server/daemon) সেটা চলছে না। সমাধান `open -a Docker`।
2. zsh আগে পুরনো kubectl এর জায়গা মনে রেখেছিল (hash); `which` প্রতিবার নতুন করে খোঁজে, কিন্তু চালানোর সময় zsh মনে রাখা জায়গা ব্যবহার করে। সমাধান `rehash` বা নতুন tab।
3. `~/.kube/config` পড়ে। Context = একটা cluster (API server এর ঠিকানা ও certificate) + একটা user (পরিচয়) + একটা default namespace।
4. Tag (`v1.37.0`) পরে অন্য image কে দেখাতে পারে; digest image এর আঙুলের ছাপ, কখনো বদলায় না — যেকোনো সময়, যেকোনো laptop এ হুবহু একই cluster।
5. Node এর IP Docker এর network থেকে (node গুলো Docker container)। Pod এর IP দেয় CNI (kindnet), প্রতিটা node কে আলাদা ভাগ দিয়ে। Service এর জন্য তৃতীয় একটা range আছে।
6. `--wait` শুধু control plane এর জন্য অপেক্ষা করে। Worker Ready হতে তার kindnet (CNI) চালু হওয়া লাগে, যেটাতে কয়েক সেকেন্ড লেগেছিল। এখন script এ `kubectl wait --for=condition=Ready nodes --all` আছে।
7. `export PATH="<folder>:$PATH"` — `~/.zshrc` এর একদম শেষে, তারপর নতুন tab।
8. Warp নিজের prompt বসিয়ে `PROMPT` বদলে দেয়। Warp এর `kube-context` chip cluster দেখায়; namespace: `kubectl config view --minify -o jsonpath='{..namespace}'; echo`।

প্রশ্নোত্তর

Setup শেষে আপনার করা প্রশ্নগুলো, উত্তর সহ — সবচেয়ে গুরুত্বপূর্ণ কথা: prompt এর নাম শুধু kubeconfig থেকে পড়া একটা লেখা, cluster চলছে কিনা তার প্রমাণ না।

`kind-config.yaml` এর মান কোথা থেকে আসে?

YAML এর কোনো field নিজে বানানো হয় না — সব আসে official docs থেকে, আমরা শুধু option বাছি।

অংশ	উৎস	কে ঠিক করে
`kind: Cluster`, `apiVersion: kind.x-k8s.io/v1alpha4`	kind Configuration docs	kind — এই format ই সে বোঝে
`role`	kind এ শুধু দুটো: `control-plane`, `worker`	kind
কয়টা node	প্রয়োজন আর RAM	আমরা
`image` + `@sha256`	নিজের kind version এর release notes	kind version + পছন্দের Kubernetes version
`extraPortMappings`	kind docs এর "Mapping ports"	আমরা — পরে কোন port লাগবে

Node layout	কখন
১টা node	শুধু YAML চালিয়ে দেখা, সবচেয়ে কম RAM
১ control-plane + ২ worker (আমাদের)	scheduling, node failure শেখা
৩ control-plane + ৩ worker	control plane এর high availability (P11/P12), অনেক RAM

Image খোঁজার ৩ ধাপ: `kind version` দেখুন → `github.com/kubernetes-sigs/kind/releases/tag/<version>` খুলুন → "Images pre-built for this release" তালিকা থেকে একটা লাইন নিন। সাধারণত সবচেয়ে নতুনটা; অফিসের production যে version এ চলে, সেটার কাছাকাছি নিলে lab আর production মেলে। Image সবসময় নিজের kind version এর release page থেকেই।

Namespace কেন — না দিলে কী হয়?

Namespace না থাকলে	উদাহরণ
একই নাম দুবার রাখা যায় না	dev আর prod দুই জায়গায় Deployment `api` — এক namespace এ অসম্ভব

Namespace না থাকলে	উদাহরণ
সব এক তালিকায়	`kubectl get pods` এ সব environment এর Pod একসাথে
ভুল জিনিস মোছার ঝুঁকি	dev পরিষ্কার করতে গিয়ে prod এর জিনিস মুছে যাওয়া
আলাদা permission দেওয়া যায় না	RBAC namespace ধরে কাজ করে (M8.1)
আলাদা resource সীমা দেওয়া যায় না	ResourceQuota namespace ধরে বসে (M5.3)
একবারে পরিষ্কার করা যায় না	`kubectl delete namespace lab` [PROD-CAUTION]

Namespace নিজে security এর দেয়াল না — দেয়াল বানাতে NetworkPolicy আর RBAC লাগে (P8)।

```
# দুই namespace এ একই নাম — চলবে; একই namespace এ দুবার — "already exists"
kubectl create deployment api --image=nginx:1.27 -n subtrack-dev
kubectl create deployment api --image=nginx:1.27 -n subtrack-prod
kubectl get deployments -A | grep api
kubectl delete deployment api -n subtrack-dev
kubectl delete deployment api -n subtrack-prod
```

`kind-subtrack-lab` মানে কী?

`(⎈|kind-subtrack-lab:lab)` = Kubernetes চিহ্ন · `kind-` (kind এর বানানো cluster, kind নিজে বসায়) + `subtrack-lab` (config এর `name`) · `:lab` (default namespace)। পুরোটার মানে: "এখন kubectl চালালে kind এর subtrack-lab cluster এর lab namespace এ যাবে।" `kind-subtrack-lab` হলো একটা **context** এর নাম।

এটা কি সবসময় চালু বা open থাকে?

না। তিনটা আলাদা জিনিস:

জিনিস	কোথায়	কখন থাকে
Prompt এর লেখা	terminal এ, শুধু লেখা	kubeconfig এ context যতদিন আছে
Context (ঠিকানা)	`~/.kube/config`	cluster না মোছা পর্যন্ত
Cluster নিজে	Docker এর ৩টা container, RAM খায়	Docker চালু থাকলে

নাম দেখা মানে contact list এ নম্বর আছে — কল চলছে না। Cluster চলছে কিনা জানতে `kubectl get nodes`।

Prompt এর লেখা আর cluster — বন্ধ ও চালু

```
# শুধু prompt এর লেখা (cluster এর কিছু বদলায় না)
kubeoff        # এই tab এ লুকাও
kubeon         # আবার দেখাও
kubeoff -g     # সব tab এ লুকাও
kubeon -g      # সব tab এ দেখাও

# Cluster নিজে (RAM খালি হয়, ভেতরের সব থেকে যায়)
docker stop $(docker ps -q --filter label=io.x-k8s.kind.cluster=subtrack-lab)
docker start $(docker ps -aq --filter label=io.x-k8s.kind.cluster=subtrack-
lab)
kubectl wait --for=condition=Ready nodes --all --timeout=180s
```

kind প্রতিটা node container এ cluster এর নাম দিয়ে label লাগায়; `--filter label=...` সেই label ধরে শুধু এই cluster এর container বাছে। আরও সহজ: Docker Desktop Quit। চালুর পর না ফিরলে `./scripts/reset-lab.sh`। নাম চিরতরে চলে যায় শুধু `kind delete cluster --name subtrack-lab` করলে।

অন্য project এর folder এও নাম দেখায় কেন?

`~/.kube/config` folder ধরে না, পুরো user ধরে কাজ করে — যেকোনো folder থেকে kubectl চালালে একই cluster এ যায়, তাই নাম দেখানো সঠিক। পথগুলো:

পথ	ফলাফল
ক. যেমন আছে	সব জায়গায় দেখায়
খ. `kubeoff -g`	কোথাও দেখায় না; দরকারে `kubeon -g`
গ. `.zshrc` এ `chpwd` hook	শুধু `~/k8s-lab` এর ভেতরে দেখায়
ঘ. Warp এর Input type → Warp	শুধু `kubectl` লেখার সময় chip দেখায়

লুকালে শুধু লেখা যায়, আচরণ না। একটাই cluster থাকলে যেকোনো পথ নিরাপদ; একাধিক cluster এলে পথ ক তে ফিরবেন।

আলাদা project এ আলাদা cluster হলে?

kubectl একসাথে একটাই current context রাখে, আর সেটা সব tab ও সব folder এর জন্য একই — একটা tab এ `kubectl config use-context <অন্য>` চালালে সব tab বদলে যায়। তখন prompt এর নামই পাহারাদার: যেকোনো `delete`, `drain`, `scale` এর আগে এক নজর। প্রতি project এ আলাদা kubeconfig বা folder অনুযায়ী নিজে থেকে context বদলানো M1.5 এ শিখব।

আরেকটা namespace বা cluster লাগলে?

Namespace: `cluster/namespaces.yaml` এ `---` দিয়ে নতুন অংশ যোগ → `kubectl apply -f cluster/namespaces.yaml` → `git commit`। File এ থাকলে reset এর পরেও ফেরে।

Cluster: নতুন config file (আলাদা `name`; `extraPortMappings` বাদ বা অন্য port — 30080/30443 আগেই নেওয়া; RAM বাঁচাতে ১টা node) → `kind create cluster --config <file>`।

```
kind get clusters                              # সব kind cluster
kubectl config get-contexts                    # সব context, * = এখনকার
kubectl config use-context kind-subtrack-lab   # বদলানো
kind delete cluster --name <নাম>               # মোছা
```

Team কে শেখানোর outline (১০ মিনিট)

মূল বার্তা: "Laptop এ ১ ঘণ্টায় একটা আসল ৩-node Kubernetes cluster চালানো যায়, আর পুরোটা Git এ রেখে এক command এ আবার বানানো যায়।"

1. **Building এর উদাহরণ (২ মিনিট):** Docker = জমি, kind = ঠিকাদার, cluster = office + ফ্ল্যাট, kubectl = ফোন, kubeconfig = contact list।
2. **Live demo (৪ মিনিট):** `docker ps` (৩টা node আসলে container) → `kubectl get nodes -o wide` → smoke test এ `Welcome to nginx!`।
3. **একটা আশ্চর্য জিনিস (২ মিনিট):** তিনটা node এর kernel একই — container আর VM এর পার্থক্য চোখে দেখা।
4. **নিরাপত্তার অভ্যাস (১ মিনিট):** prompt/chip এ cluster দেখে command, version pin (digest), secret Git এ না।
5. **প্রশ্ন (১ মিনিট):** "Docker বন্ধ করলে কী হবে?" — শ্রোতারা উত্তর দিক।

Sources

- kind Quick Start

- kind v0.33.0 release notes — default node image v1.37.0 ও digest
- Install kubectl on Linux — version skew নিয়ম
- Installing Helm — `get-helm-4` script
- Install Docker Desktop on Mac — macOS support নীতি
- Homebrew Support Tiers
- Warp: Terminal prompt — Kubernetes context chip
- Kubernetes Zero-to-Advanced Roadmap — C+ Setup অংশ